

รายงานโครงงานวิชา Machine Learning

การทำนายจำนวนการเช่าจักรยานรายชั่วโมง ในกรุงโซลจากสภาพอากาศและปฏิทิน

Seoul Bike Sharing Demand — Regression

รหัสனிสิต	ชื่อ-นามสกุล
6330300160	นาย ชญานิน ตลับเงิน
6730300591	นาย ศุภวิชญ์ ไชยานุศักดิ์
6730300850	นาย พัฒนเดช มะหิเมือง

ตุลาคม 2569

github.com/Sh080205/seoul-bike-demand

สารบัญ

ตารางเทียบกับข้อกำหนดของโครงการ

บทคัดย่อ

1. บทนำและนิยามปัญหา
2. ข้อมูล
3. การแบ่งข้อมูลและการป้องกัน leakage
4. การสำรวจข้อมูล (EDA)
5. Feature engineering และ preprocessing
6. Feature selection
7. PCA
8. Baseline และการเทียบโมเดล
9. Hyperparameter tuning
10. การเลือกโมเดลและทรัพยากรที่ใช้
11. ผลบนชุดทดสอบและ error analysis
12. การทำนายข้อมูลใหม่
13. ความท้าทายที่พบ
14. อภิปรายผล ข้อจำกัด และแนวทางพัฒนาต่อ
15. สรุป

ภาคผนวก ก. เซตลิสต์ป้องกัน leakage · ข. การรันซ้ำ

เอกสารอ้างอิง

ตารางเทียบกับข้อกำหนดของโครงการ

ข้อกำหนด	สิ่งที่ทำ	หัวข้อ
1. เลือก dataset และกำหนดเป้าหมาย	Seoul Bike Sharing Demand (UCI) · regression ทำนายยอดเช่าคัน/ชม.	1, 2
1. EDA: สถิติเชิงพรรณนา, visualization, ความสัมพันธ์, outlier	<code>describe()</code> , 7 กราฟ, Pearson correlation, กฎ IQR — ทำบน train เท่านั้น	4
2. จัดการ missing value	ความถี่ 0% (ค่าผิดของเซนเซอร์) → NaN → เติมด้วย median ใน Pipeline	4.4, 5
2. จัดการ outlier	ตรวจด้วย IQR แล้วเก็บไว้ เพราะเป็นเหตุการณ์จริง	4.4
2. Encoding	One-hot สำหรับฤดู เดือน วัน · ชั่วโมงเลือก encoding ตามชนิดโมเดล	5
2. Feature scaling	StandardScaler (Z-score) พร้อมเหตุผลที่ไม่ใช้ Min-Max	5
3. Filter method	<code>f_regression</code> (correlation) และ <code>mutual_info_regression</code> · อธิบายว่าทำไมไม่ใช้ χ^2 ไม่ได้	6
3. Wrapper method	RFE	6
3. Embedded method	Lasso (L1)	6
3. Feature extraction	PCA	7
4. แบ่ง train/test 70–80 / 20–30	80 / 20 ตามวัน (292 / 73 วัน) + 5-fold GroupKFold บน train	3
4. อัลกอริทึมอย่างน้อย 3–4 แบบ	Linear Regression, kNN, Random Forest, HistGradientBoosting + baseline 3 แบบ	8, 9
4. เมตริก MAE, MSE, RMSE, R^2	ครบทั้ง 4 ค่า ทั้งใน CV ของทุกโมเดล (ตารางที่ 8.1) และบน test (ตารางที่ 11.1) · MAE เป็นเมตริกหลัก	1, 8, 11
4. วิเคราะห์ overfitting / underfitting	คอสมันน์ gap = CV MAE – MAE บน train (ตารางที่ 8.1)	8
4. Model selection	เลือกด้วย CV MAE + ทฤษฎีการใช้ (ไม่ใช่ test)	10
ป้องกัน data leakage	split และ CV ตามวัน · ทุกขั้นที่เรียนจากข้อมูลอยู่ใน Pipeline · พิสูจน์ว่า fold (ตารางที่ 5.3)	3, 5, ภาคผนวก ก
5. สรุปโมเดลที่ดีที่สุดและประสิทธิภาพ	HistGradientBoosting · ผล test และ error analysis	10, 11, 15
5. ทำนายข้อมูลใหม่	Pipeline ที่ใช้งานจริง + 4 สถานการณ์สมมติ พร้อมแปลความหมาย	12
ความท้าทายที่พบ	ตารางปัญหาและวิธีแก้	13
สิ่งที่ส่ง: รายงาน · โค้ด · สไลด์	รายงานนี้ (reports/report.pdf) · notebook 00_setup, 01_eda, 02_modeling พร้อมคำอธิบายทุก cell · สไลด์นำเสนอ 18 หน้า (slides/seoul-bike-presentation.html) และบทพูด (slides/speaker-script.html)	ภาคผนวก ข

บทคัดย่อ

โครงการนี้สร้างโมเดล regression ทำนาย **จำนวนจักรยานที่ถูกเช่าในแต่ละชั่วโมง** ของระบบ Seoul Bike ทั้งเมือง จากข้อมูลที่เราสามารถหาได้ คือพยากรณ์อากาศและปฏิทิน โดยใช้ชุดข้อมูล Seoul Bike Sharing Demand จาก UCI (8,760 ชั่วโมง ธ.ค. 2017 – พ.ย. 2018) ข้อมูลถูกแบ่ง train/test **ตามวัน** เพื่อไม่ให้ชั่วโมงของวันเดียวกันรั่วข้ามสองฝั่ง และทุกการตัดสินใจใช้ 5-fold GroupKFold บน train เท่านั้น เราเทียบ 4 ตระกูลโมเดล (Linear Regression, kNN, Random Forest, HistGradientBoosting) กับ baseline 3 แบบ พร้อมทำ feature selection 3 แนวทางและ PCA โมเดลที่เลือกคือ **HistGradientBoostingRegressor** ได้ CV MAE 109.0 ± 8.1 คัน/ชม. และเมื่อประเมินชุดทดสอบ ครั้งเดียวได้ MAE 97.4 คัน/ชม., MSE 28,947.7 (RMSE 170.1 คัน/ชม.) และ R^2 0.919 จุดอ่อนหลักคือชั่วโมงที่ฝนตก และเหตุการณ์พิเศษที่ไม่มีใน feature เช่น พายุไต้ฝุ่นและเทศกาล

109.0

CV MAE (คัน/ชม.)
5-fold GroupKFold

97.4

test MAE (คัน/ชม.)
ประเมินครั้งเดียว

0.919

test R^2

-73%

MAE เทียบ baseline
ที่ใช้แค่เวลา (401.0)

1. บทนำและนิยามปัญหา

ระบบเช่าจักรยานสาธารณะในเมืองใหญ่ต้องมีจักรยานพร้อมให้เช่าในชั่วโมงที่คนต้องการ ถ้าจักรยานไม่พอ คนจะเช่าไม่ได้ ถ้าเตรียมไว้มากเกินไปก็เสียแรงขนย้ายโดยไม่จำเป็น การรู้ล่วงหน้าว่าแต่ละชั่วโมงจะมีคนเช่ากี่คันจึงช่วยให้ผู้ให้บริการวางแผนได้ โครงการนี้ใช้ข้อมูลของระบบ Seoul Bike กรุงโซล ซึ่งบันทึกยอดเช่ารายชั่วโมงคู่กับสภาพอากาศ มาสร้างโมเดลที่ทำนายยอดเช่าจากข้อมูลที่เราหาได้ คือพยากรณ์อากาศและปฏิทิน และทำตามขั้นตอนของโจทย์ครบ ตั้งแต่การสำรวจข้อมูล การเตรียมข้อมูล การคัดเลือกและสกัด feature การสร้างและเทียบ 4 อัลกอริทึม การเลือกโมเดล ไปจนถึงการทำนายข้อมูลใหม่

หัวข้อ	รายละเอียด
ชนิดงาน	Regression
ทำนายอะไร	จำนวนจักรยานที่ถูกเช่าในชั่วโมงนั้น (rented_bike_count , คัน/ชั่วโมง) ของทั้งระบบ
ใช้ทำอะไร	ให้ผู้ให้บริการวางแผนจำนวนจักรยานที่ต้องพร้อมใช้ และจัดคนกระจายจักรยานล่วงหน้าในชั่วโมงที่คาดว่าจะมีคนเช่ามาก
input	สภาพอากาศ (อุณหภูมิ, ความชื้น, ลม, ทิศนํ้า, dew point, แสงแดด, ฝน, หิมะ) + เวลา (ชั่วโมง, วันในสัปดาห์, เดือน) + ฤดู + วันหยุด
เมตริกหลัก	MAE (คัน/ชม.) — ดีความตรงว่า "หายคลาดเคลื่อนกี่คัน" และไม่ถูกชั่วโมงผิดไม่กี่ชั่วโมงครอบงำเหมือน RMSE (target เบ้า: mean 756.8 > median 582.5)
เมตริกรอง	MSE และ RMSE (ดูว่ามี error ก้อนใหญ่ไหม) และ R^2 (สัดส่วนความแปรปรวนที่อธิบายได้) · รายงาน MSE ตามเกณฑ์ แต่อธิบายผลด้วย RMSE เพราะ $MSE = RMSE^2$ มีหน่วยเป็น "คัน ² " ซึ่งตีความไม่ได้โดยตรง

ขอบเขต: ทำนายเฉพาะชั่วโมงที่ระบบเปิดให้บริการ (Functioning Day = Yes) · ระดับทั้งเมือง ไม่ใช่รายสถานี · ไม่ใช่ยอดเช่าของชั่วโมงก่อนหน้า (lag) เพราะเป็นการใช้ target ทำนาย target · โจทย์คือ "อากาศ + เวลาแบบนี้ → จะมีคนเช่ากี่คัน" ไม่ใช่การพยากรณ์อนุกรมเวลา

ทำไมเลือก **regression**: target ของชุดข้อมูลเป็นจำนวนนับต่อเนื่อง ทำนายได้ตรงๆ โดยไม่ต้องตั้ง threshold เอง และตัวเลข "ก็คั่น" นำไปวางแผนได้ทันที

2. ข้อมูล

► วิธีดำเนินการ

เลือกชุดข้อมูล **Seoul Bike Sharing Demand** (UCI Machine Learning Repository, id 560, สัญญาอนุญาต CC BY 4.0) ซึ่งบันทึกยอดเช่ารายชั่วโมงของระบบจักรยานสาธารณะกรุงโซล ตั้งแต่ 1 ธ.ค. 2017 ถึง 30 พ.ย. 2018 จากนั้นตรวจ byte ดิบของไฟล์เพื่อเลือก encoding ตรวจรูปแบบวันที่ เปลี่ยนชื่อคอลัมน์เป็น snake_case แล้วตรวจสอบความครบถ้วน แยกค่าที่เป็นไปได้ของคอลัมน์หมวดหมู่

► ผลลัพธ์

ตารางที่ 2.1 ตัวแปรในชุดข้อมูล (เปลี่ยนชื่อเป็น snake_case)

คอลัมน์ดิบ	ชื่อที่ใช้	ชนิด / หน่วย
Date	date	วันที่ (dd/mm/yyyy) — ใช้เป็น group ของการ split ไม่ใช่ feature
Rented Bike Count	rented_bike_count	target · คั่น/ชั่วโมง
Hour	hour	0–23
Temperature(°C)	temp_c	°C
Humidity(%)	humidity_pct	%
Wind speed (m/s)	wind_speed_ms	m/s
Visibility (10m)	visibility_10m	หน่วย 10 เมตร (27–2,000)
Dew point temperature(°C)	dew_point_c	°C
Solar Radiation (MJ/m2)	solar_radiation_mj	MJ/m ²
Rainfall(mm)	rainfall_mm	มม.
Snowfall (cm)	snowfall_cm	ซม.
Seasons	seasons	Winter / Spring / Summer / Autumn
Holiday	holiday	Holiday / No Holiday
Functioning Day	functioning_day	Yes / No (ระบบเปิด/ปิด)

ตารางที่ 2.2 ตรวจสอบไฟล์ดิบก่อนโหลดจริง (01_eda ชั้น 2.1)

ตรวจ	ผล
อ่านหัวตารางด้วย utf-8	UnicodeDecodeError ที่ byte \xb0 (สัญลักษณ์ °)
อ่านด้วย cp1252	Temperature(°C) ถูกต้อง → ใช้ค่านี้ (บันทึกใน config.py)
อ่านด้วย latin-1	Temperature(°C) ถูกต้องเช่นกัน
วันที่ ส่วนที่ 1 / 2 / 3	1-31 / 1-12 / 2017-2018 → วัน/เดือน/ปี (01/12/2017 = 1 ธ.ค. 2017)

ไฟล์ไม่ใช่ UTF-8 (สัญลักษณ์ ° เก็บเป็น byte 0xB0) จึงอ่านด้วย cp1252 และรูปแบบวันที่ %d/%m/%Y เอง (ตรวจจากค่าส่วนแรกที่ขึ้นถึง 31) ได้ **8,760 แถว × 14 คอลัมน์** ครบ 365 วัน × 24 ชั่วโมง ไม่มี missing value และไม่มีแถวซ้ำ

► การวิเคราะห์และการตัดสินใจ

- target เป็นจำนวนนับต่อเนื่อง และ feature ทั้งหมดเป็นข้อมูลที่เราสามารถหาได้จากพยากรณ์อากาศและปฏิทิน จึงเหมาะกับ โจทย์ regression ที่นำไปใช้วางแผนได้จริง
- การที่ไม่มี missing value ในไฟล์ไม่ได้แปลว่าข้อมูลถูกทุกค่า ชั้น EDA จึงตรวจค่าที่เป็นไปไม่ได้ทางกายภาพต่อ (พบ ความชื้น 0% ในหัวข้อ 4.4)
- date ไม่ใช่เป็น feature โดยตรง แต่ใช้เป็นกลุ่มของการแบ่งข้อมูล (หัวข้อ 3) และใช้สร้าง feature ปฏิทิน (หัวข้อ 5)

3. การแบ่งข้อมูลและการป้องกัน leakage

► วิธีดำเนินการ

ชั่วโมงติดกันในวันเดียวกันมีอากาศและยอดเข้าใกล้กันมาก ถ้าสุ่มแบ่งทีละแถว ชั่วโมง 14:00 อาจอยู่ใน train ส่วน 15:00 อยู่ใน test โมเดลจะแค่ "จำวันนั้น" ได้ ทำให้คะแนนสูงเกินจริง จึงใช้ **วันที่เป็นกลุ่ม (group)** ให้ทั้ง 24 ชั่วโมงของวันหนึ่งอยู่ฝั่งเดียวกันเสมอ โดยแบ่งด้วย GroupShuffleSplit เป็น train 80% / test 20% ของจำนวนวัน ก่อน ทำ EDA และบันทึก รายการวันเป็นไฟล์ จากนั้นทุกชั้นที่ต้องประเมินโมเดลบน train ใช้ 5-fold GroupKFold

► ผลลัพธ์

ตารางที่ 3.1 ผลการแบ่งด้วย GroupShuffleSplit (test_size = 0.2 ของจำนวนวัน, random_state = 42)

	วัน	แถว	ชั่วโมงที่ระบบปิด	วันหยุดนักขัตฤกษ์	สัดส่วนฤดูหนาว
train	292	7,008	240	12	22.9%
test	73	1,752	55	6	31.5%

เมื่อทดลองกับ KFold แบบสุ่มแถว validation fold มีแถวจาก 281 วัน และ **ทุกวันนี้ชั่วโมงอื่นของวันเดียวกันอยู่ใน train** ส่วน GroupKFold มีวันซ้ำระหว่าง train กับ validation 0 วันในทุก fold

► การวิเคราะห์และการตัดสินใจ

- เลือกสุ่มวันจากทั้งปี (แทนการตัดตามเวลา) เพราะข้อมูลมีแค่ 1 ปี ถ้าตัดตามเวลา test จะมีแค่ 1-2 ฤดูที่โมเดลไม่เคยเห็น
- test มีสัดส่วนฤดูหนาวสูงกว่า train จากความแปรปรวนของการสุ่มเพียง 73 วัน เราไม่สุ่มใหม่จนกว่าตัวเลขจะ "สวย" เพราะนั่นคือการเลือก test ด้วยมือ แต่บันทึกไว้เพื่อใช้ตีความผล test (หัวข้อ 11)

- รายการวันถูกบันทึกเป็นไฟล์ `data/splits/train_dates.csv`, `test_dates.csv` และ commit ไว้ทุก notebook ใช้วัน test ชุดเดียวกัน
- ผลทดลอง KFold ยืนยันว่าการสุ่มที่ละแถวทำให้ข้อมูลของวันเดียวกันรั่วข้าม fold จริง จึงกำหนดให้ทุกชั้นที่มี cross-validation ใช้ **5-fold GroupKFold** และส่ง `groups` = วันที่ทุกครั้ง

4. การสำรวจข้อมูล (EDA) — ใช้ train เท่านั้น

► วิธีดำเนินการ

สำรวจเฉพาะ train (7,008 แถว, 292 วัน) เพื่อไม่ให้ข้อมูล test มีผลต่อการตัดสินใจใดๆ โดยใช้สถิติเชิงพรรณนา (`describe()`, skewness), กราฟ (histogram, boxplot, line plot รายชั่วโมง, กราฟตามฤดู อุณหภูมิ และฝน), Pearson correlation และกฎ IQR สำหรับค่าผิดปกติ ทุกหัวข้อจบด้วยการตัดสินใจที่ส่งต่อไปยังขั้นเตรียมข้อมูล (หัวข้อ 5)

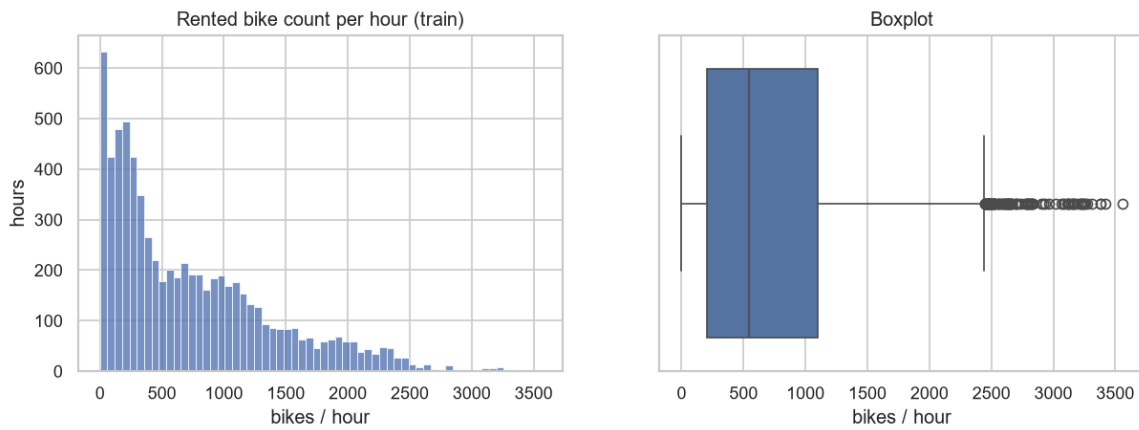
4.1 สถิติเชิงพรรณนา ชั่วโมงที่ระบบปิด และ distribution ของ target

► ผลลัพธ์

ตารางที่ 4.1 สถิติเชิงพรรณนาของตัวแปรตัวเลข (train 7,008 แถว ก่อนตัดชั่วโมงที่ระบบปิด)

ตัวแปร	mean	std	min	Q1	median	Q3	max
<code>rented_bike_count</code>	730.86	653.78	0	206	547	1,101	3,556
<code>temp_c</code>	13.46	11.72	-16.2	4.10	14.80	22.80	39.4
<code>humidity_pct</code>	58.56	20.35	0	43	58	74.25	98
<code>wind_speed_ms</code>	1.70	1.02	0	0.90	1.50	2.30	7.4
<code>visibility_10m</code>	1,430.96	601.10	27	943.75	1,662	1,998	2,000
<code>dew_point_c</code>	4.71	12.85	-30.6	-4.20	5.95	15.40	27.2
<code>solar_radiation_mj</code>	0.58	0.88	0	0	0.01	0.95	3.52
<code>rainfall_mm</code>	0.15	1.16	0	0	0	0	35.0
<code>snowfall_cm</code>	0.07	0.45	0	0	0	0	8.8

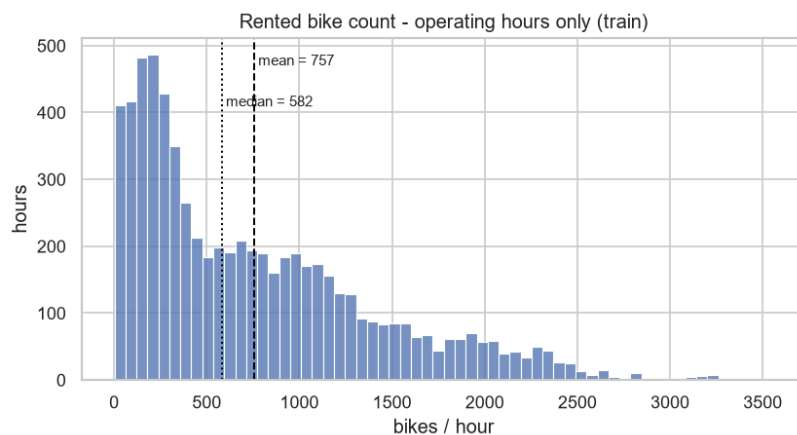
- ยอดเช่าเบ้ขวา (mean 730.9 > median 547) และมีค่า 0 · `visibility_10m` มีสเกลใหญ่กว่าตัวแปรอื่นมาก · ฝน หิมะ และแสงแดดส่วนใหญ่เป็น 0 (zero-inflated) · ความชื้นต่ำสุด **0%** ซึ่งเป็นไปไม่ได้ทางกายภาพ (ตรวจสอบในหัวข้อ 4.4)



รูปที่ 4.1 histogram และ boxplot ของยอดเช่ารายชั่วโมง (train ทั้งหมด ก่อนตัดชั่วโมงที่ระบบปิด)

histogram เบ้ขวาชัดเจน (skewness 1.08) แ่งแรกใกล้ 0 สูงที่สุด ส่วนหนึ่งมาจาก 240 แถว (3.4%) ที่ยอดเป็น 0 พอดี และ boxplot มีจุดเกินหนดขวา ($> 2,443.5$ คัน) 102 แถว

ใน train มี 240 แถวที่ `functioning_day = No` (ระบบปิดทั้งวัน 10 วัน) ยอดเช่าเป็น 0 ทุกแถว และแถวที่ยอดเป็น 0 ทั้งหมด คือแถวเหล่านี้ หลังตัดออกเหลือ **6,768 แถว (282 วัน)** ยอดต่ำสุด 2 คัน mean 756.8 median 582.5 skewness 1.07 (เบ้ขวา)



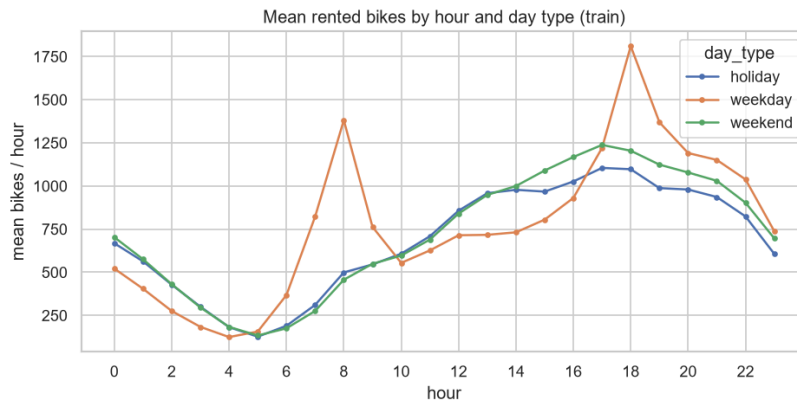
รูปที่ 4.2 distribution ของยอดเช่าหลังตัดชั่วโมงที่ระบบปิด พร้อมเส้น mean และ median

► การวิเคราะห์และการตัดสินใจ

- ถ้าเก็บชั่วโมงที่ระบบปิดไว้ โมเดลจะได้คะแนนฟรีจากกฎ "ปิด $\rightarrow 0$ " ที่ผู้ให้บริการรู้อยู่แล้ว จึง **ตัดออกทั้ง train และ test** (เป็นกฎนิยามขอบเขต ไม่ได้เรียนค่าจากข้อมูล จึงไม่รั่ว)
- target เบ้ขวา (mean สูงกว่า median $\sim 30\%$) ชั่วโมงพิคไม่กี่ชั่วโมงจะครอบงำ RMSE จึงใช้ **MAE เป็นเมตริกหลัก** และจะลองแปลง `log1p(y)` กับโมเดลเชิงเส้นในขั้น tuning

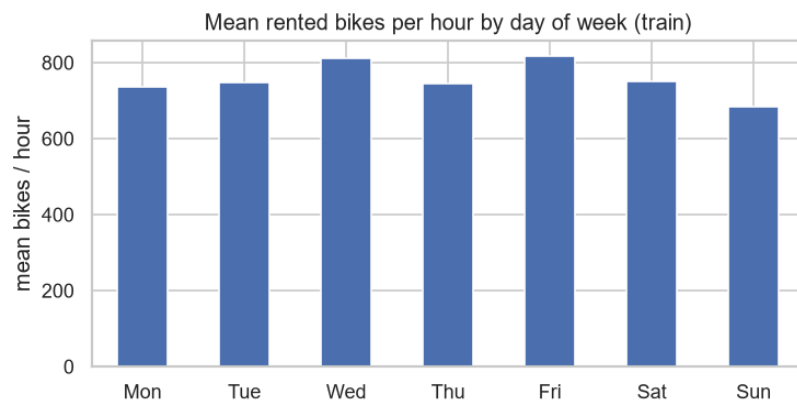
4.2 รูปแบบรายชั่วโมงและประเภทวัน

► ผลลัพธ์



รูปที่ 4.3 ยอดเช่าเฉลี่ยรายชั่วโมง แยกวันทำงาน / เสาร์-อาทิตย์ / วันหยุดนักขัตฤกษ์

วันทำงานมีพีคแหลม 2 ครั้งที่ **8:00** (~1,380 คัน) และ **18:00** (~1,810 คัน) ซึ่งเป็นการเดินทางไป-กลับที่ทำงาน ส่วนเสาร์-อาทิตย์และวันหยุดนักขัตฤกษ์มีรูปร่างเดียวกัน คือไม่มีพีคเช้าและสูงสุดราว 17:00 ทุกประเภทวันต่ำสุดตอน 4:00-5:00



รูปที่ 4.4 ยอดเช่าเฉลี่ยต่อชั่วโมงแยกตามวันในสัปดาห์

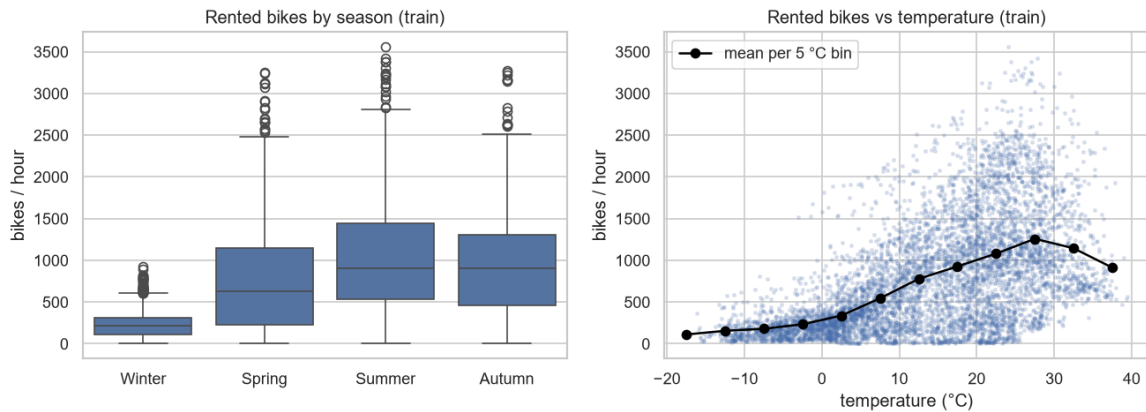
เมื่อเฉลี่ยทั้งวัน แต่ละวันในสัปดาห์ต่างกันไม่มาก (~680-820 คัน/ชม.) อาทิตย์ต่ำสุด (682) พุธและศุกร์สูงสุด (811, 816)

► การวิเคราะห์และการตัดสินใจ

- ชั่วโมงสัมพันธ์กับยอดเช่าแบบ **ไม่เป็นเส้นตรงและเป็นวงรอบ** (23:00 อยู่ติดกับ 0:00) → สร้าง `hour_sin`, `hour_cos`
- รูปร่างเส้นขึ้นกับประเภทวัน (interaction ชั่วโมง × วันหยุด) → สร้าง `is_weekend`, `is_holiday` · โมเดลเชิงเส้นจับ interaction เองไม่ได้ ส่วนโมเดลต้นไม้จับได้ จึงคาดว่าโมเดลต้นไม้จะได้เปรียบ
- sin/cos คู่เดียวเป็นคลื่นลูกเดียวต่อวัน จับ 2 พีคไม่ได้ → สำหรับโมเดลเชิงเส้นต้องลอง one-hot ของชั่วโมงด้วย
- รูปที่ 4.3 กับ 4.4 รวมกันบอกว่าความต่างของวันหยุดกับวันทำงานอยู่ที่ **"รูปร่าง"** ของเส้นรายชั่วโมง ไม่ใช่ "ระดับ" ของทั้งวัน → สร้างทั้ง `day_of_week` และ `is_weekend` แล้วให้ feature selection ตัดสินว่าตัวไหนมีประโยชน์

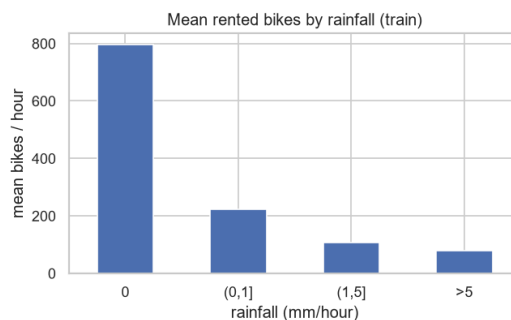
4.3 ฤดู อุณหภูมิ และฝน

► ผลลัพธ์



รูปที่ 4.5 ยอดเช่าตามฤดู (ซ้าย) และตามอุณหภูมิพร้อมค่าเฉลี่ยทุกช่วง 5 °C (ขวา)

- ฤดูหนาวเฉลี่ยเพียง ~230 คัน/ชม. เทียบกับ 774–1,037 ในฤดูอื่น
- ยอดเพิ่มตามอุณหภูมิจนสูงสุดที่ 25–30 °C (~1,258) แล้ว **ลดลง** เมื่อร้อนเกิน 30 °C
- ฝนเป็น 0 ใน 93.9% ของชั่วโมง แต่ชั่วโมงที่ฝนตกยอดตกจาก ~795 เหลือ ~164 คัน และแม้ฝนแค่ ≤ 1 มม. ยอดก็เหลือ ~223



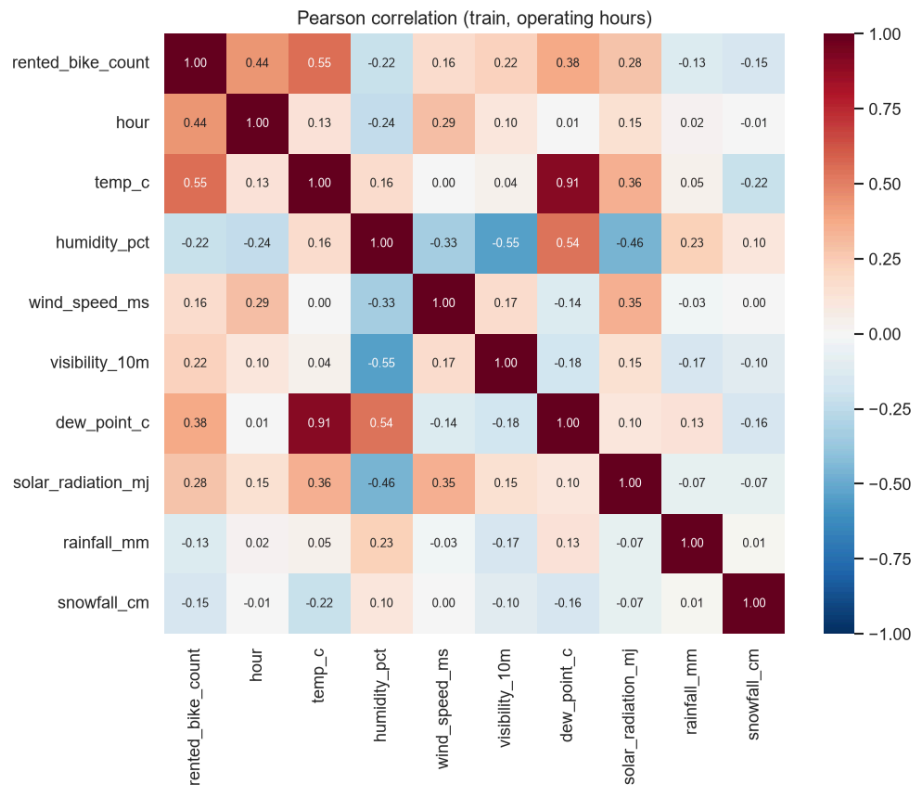
รูปที่ 4.6 ยอดเช่าเฉลี่ยตามปริมาณฝน

► การวิเคราะห์และการตัดสินใจ

- อุณหภูมิกับยอดเช่าเป็นความสัมพันธ์ **โค้งคว่ำ** ซึ่งโมเดลเชิงเส้นจับไม่ได้ เป็นอีกเหตุผลที่คิดว่าโมเดลต้นไม้จะแม่นยำกว่า
- ผลของ **"มีฝนหรือไม่"** สำคัญกว่าปริมาณฝน → สร้าง `is_raining` · ฝนเป็นเหตุการณ์ที่มีน้อย (~6%) จึงคาดว่าจะป็นจุดที่โมเดลหายยาก
- เก็บทั้ง `seasons` และสร้าง `month` ที่ละเอียดกว่า แล้วให้ชั้น feature selection ตัดสินว่าเข้าช้อนหรือไม่

4.4 Correlation และ outlier

▶ ผลลัพธ์



รูปที่ 4.7 Pearson correlation ของตัวแปรตัวเลข (train, เฉพาะชั่วโมงที่ระบบเปิด)

- มีคู่ตัวแปรเดียวที่ $|r| > 0.8$ คือ `temp_c` กับ `dew_point_c` ($r = 0.91$) · กับ target: `temp_c` $r = 0.55$, `hour` $r = 0.44$ · ฝน/หิมะมี r ต่ำ (-0.13 , -0.15) ทั้งที่หัวข้อ 4.3 แสดงว่ามีผลแรง · คู่ที่สัมพันธ์ปานกลาง เช่น ความชื้นกับทัศนวิสัย (-0.55) และความชื้นกับ dew point (0.54)
- กฎ IQR (ตารางที่ 4.2) นับทุกชั่วโมงที่ฝนหรือหิมะตกเป็น "outlier" (เพราะ $IQR = 0$) และพบค่าเกินขอบใน `solar_radiation_mj` (7.18%), `wind_speed_ms` (1.63%) และยอดเขา (1.40%) ซึ่งล้วนอยู่ในช่วงที่เกิดขึ้นได้จริง
- ความชื้น = 0%** มี 17 แกว (19–28 พ.ค. 2018) เมื่อคำนวณย้อนจากอุณหภูมิและ dew point ด้วยสูตร Magnus ได้ 33–72%

ตารางที่ 4.2 outlier ตามกฎ IQR (ต่ำกว่า $Q1 - 1.5 \cdot IQR$ หรือสูงกว่า $Q3 + 1.5 \cdot IQR$) · train เฉพาะชั่วโมงที่ระบบเปิด · ตัวแปรอื่นไม่มีค่าเกินขอบ

ตัวแปร	ขอบล่าง	ขอบบน	max	outlier (แถว)	%
solar_radiation_mj	-1.42	2.38	3.52	486	7.18
rainfall_mm	0.00	0.00	35.00	413	6.10
snowfall_cm	0.00	0.00	8.80	296	4.37
wind_speed_ms	-1.20	4.40	7.40	110	1.63
rented_bike_count	-1,116	2,460	3,556	95	1.40

► การวิเคราะห์และการตัดสินใจ

- r ของฝน/หิมะต่ำเพราะความสัมพันธ์ไม่เป็นเส้นตรง (ส่วนใหญ่เป็น 0) ไม่ได้แปลว่าไม่สำคัญ → ยืนยันการสร้าง `is_raining` · **ยังไม่ตัด** `dew_point_c` แต่ให้ feature selection ตัดสิน และความซ้ำซ้อนของตัวแปรอากาศเป็นเหตุผลที่ลอง PCA
- **ไม่ตัด outlier** ของสภาพอากาศและยอดเขา เพราะเป็นเหตุการณ์จริง (ฝนหนัก, ลมแรง, ชั่วโมงพิก) ที่โมเดลต้องเจอตอนใช้งาน และชั่วโมงพิกคือชั่วโมงที่ผู้ให้บริการต้องการให้ทายแม่นยำที่สุด
- ความชื้น 0% ซัดกับอุณหภูมิและ dew point ในแถวเดียวกัน จึงเป็น **ค่าผิดของเซนเซอร์** → แปลงเป็น NaN แล้วเติมด้วยค่ามัธยฐานที่เรียนจาก training fold ใน Pipeline (หัวข้อ 5)

5. Feature engineering และ preprocessing

► วิธีดำเนินการ

1. **กฎกำหนดขอบเขต** (ใช้กับ train และ test เหมือนกัน เพราะเป็นกฎตายตัวรายละเอียด ไม่ได้เรียนค่าจากข้อมูล): ตัดแถว `functioning_day = No` และแปลง `humidity_pct = 0` เป็น NaN
2. **สร้าง feature ใหม่** จากค่าในแถวเดียวกันเท่านั้น ไม่ใช้ target (ตารางที่ 5.1)
3. **ประกอบ Pipeline**: ทุกขั้นที่เรียนค่าจากข้อมูลอยู่ใน Pipeline เพื่อให้ CV fit ใหม่ทุก fold — ตัวแปรตัวเลขและ binary ผ่าน `SimpleImputer(median)` → `StandardScaler` ส่วน `seasons`, `month`, `day_of_week` ผ่าน `OneHotEncoder` ตามด้วยช่อง `select` (สำหรับ selector/PCA) และช่อง `model`

ตารางที่ 5.1 feature ที่สร้างเพิ่ม (คำนวณจากแถวเดียวกันเท่านั้น ไม่ใช้ target)

feature	สร้างจาก	เหตุผล (จาก EDA)
<code>hour_sin</code> , <code>hour_cos</code>	<code>sin/cos(2π·hour/24)</code>	ชั่วโมงเป็นวงรอบ: 23:00 กับ 0:00 ห่างกัน 0.261 บนวงกลม เท่ากับ 0:00–1:00 (ตัวเลขติดห่าง 23)
<code>is_weekend</code> , <code>day_of_week</code>	<code>date</code>	รูปร่างรายชั่วโมงของวันหยุดต่างจากวันทำงาน
<code>is_holiday</code>	<code>holiday</code>	แปลงข้อความเป็น 0/1
<code>month</code>	<code>date</code>	ละเอียดกว่าฤดู (ต้น/ปลาย Spring ต่างกันมาก)
<code>is_raining</code>	<code>rainfall_mm > 0</code>	"มีฝน" ลดยอด ~80% ไม่ว่าจะตกมากหรือน้อย

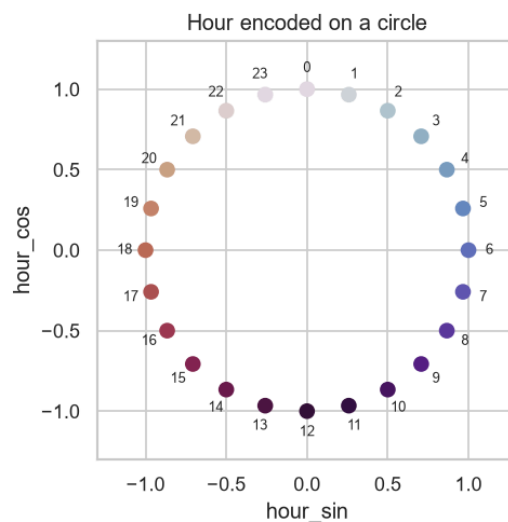
► ผลลัพธ์

- การ encode ชั่วโมงบนวงกลม: 23:00 กับ 0:00 ห่างกัน 0.261 เท่ากับ 0:00–1:00 ส่วน 0:00 กับ 12:00 ห่างกันมากที่สุด (2.0) ตรงกับธรรมชาติของเวลา (รูปที่ 5.1)
- กฎขอบเขต: train 7,008 → **6,768 แถว** (ตัด 240), test 1,752 → **1,697 แถว** (ตัด 55) · ความชื้นกลายเป็น NaN ใน train 17 แถว
- `X_train` มี 17 คอลัมน์ · หลัง preprocess (hour แบบ sin/cos) ได้ 36 คอลัมน์ ไม่มี NaN เหลือ ค่าที่ใช้เดิมความชื้น = 57.0 (มัธยฐานของ train) ตัวแปรตัวเลขทุกตัวมี mean 0 และ std 1 และ one-hot ของแต่ละกลุ่มรวมกันได้ 1 ทุกแถว
- ตรวจสอบการรั่วของ Pipeline ใน `GroupKFold` (ตารางที่ 5.3): ทุก fold มีวันซ้ำระหว่าง train กับ validation 0 วัน และค่าที่ scaler / imputer เรียนได้เปลี่ยนไปตาม fold (mean ของ `temp_c` 13.348–13.395, median ความชื้น 57–58) แปลว่า fit ใหม่จาก training fold ของตัวเองทุกรอบ ไม่เคยเห็นข้อมูล validation

ตารางที่ 5.3 จำลอง cross-validation ทีละ fold (5-fold GroupKFold บน train) เพื่อพิสูจน์ว่า Pipeline ไม่รั่ว

fold	วัน train	วัน val	วันซ้ำ	scaler mean temp_c	median ความชื้น	ยอดเฉลี่ย val
0	225	57	0	13.395	57.0	783.931
1	225	57	0	13.385	57.0	719.159
2	226	56	0	13.348	58.0	772.452
3	226	56	0	13.362	58.0	778.076
4	226	56	0	13.395	57.0	730.462

เทียบ KFold ธรรมดา fold 0: validation มี 281 วัน และซ้ำกับ train ครบทั้ง 281 วัน



รูปที่ 5.1 ชั่วโมงหลังแปลงเป็น (hour_sin, hour_cos) เรียงเป็นวงกลม

ตารางที่ 5.2 การ encode ชั่วโมงตามชนิดโมเดล

โมเดล	encoding	คอลัมน์หลัง preprocess	เหตุผล
Linear Regression	one-hot 24 คอลัมน์	58	ให้แต่ละชั่วโมงมีค่าคงที่ของตัวเอง รองรับ 2 พิค
kNN	sin/cos	36	ใช้ระยะทาง ต้องการให้ 23:00 ใกล้ 0:00 และมีดีนอย
Random Forest, HistGradientBoosting	0-23 ตรงๆ	35	ต้นไม้ตัดสินใจแบ่งช่วงชั่วโมงได้เองหลายครั้ง

► การวิเคราะห์และการตัดสินใจ

- **Missing value:** เติมด้วย **มัธยฐาน** แทนค่าเฉลี่ย เพราะไม่ไวต่อค่าสุดโต่ง และเติมใน Pipeline เพื่อให้ค่ามัธยฐานมาจาก training fold เท่านั้น ถ้าเติมก่อนแบ่งข้อมูล ค่าจาก test จะรั่วเข้ามา · ไม่ลบแถวทิ้งเพราะค่าอื่นในแถวยังถูกต้อง
- **Encoding:** ใช้ one-hot กับ `seasons`, `month`, `day_of_week` เพราะเป็นหมวดหมู่ที่ไม่มีลำดับ ถ้าใช้ label encoding (เช่น Winter = 3, Autumn = 0) โมเดลเชิงเส้นและ kNN จะตีความว่าเป็นระยะทางหรือขนาดที่ไม่มีอยู่จริง
- **ชั่วโมง** encode ต่างกันตามชนิดโมเดล เพราะ sin/cos คู่เดียวให้โมเดลเชิงเส้นสร้างพิกได้แค่ 1 ครั้งต่อวัน แต่ข้อมูลมี 2 พิค (ตารางที่ 5.2)

- **Scaling:** จำเป็นกับ kNN (ระยะทาง), PCA (variance) และ Lasso/Ridge (ขนาดของ penalty) เพราะตัวแปรที่มีสเกลต่างกันมาก เช่น `visibility_10m` 27–2,000 เทียบกับ `wind_speed_ms` 0–7.4 ส่วนโมเดลต้นไม้ไม่ได้รับผลจาก scaling แต่ใช้ Pipeline เดียวกันเพื่อให้เทียบกันได้
- **ทำไมเลือก Standardization (Z-score) แทน Min-Max:** Min-Max ใช้ค่า min และ max เป็นตัวหาร จึงไวต่อค่าสุดโต่งที่เราตัดสินใจเก็บไว้ (หัวข้อ 4.4) เช่น `rainfall_mm` ที่ max 35 มม. แต่ $Q3 = 0$ ค่าผันเกื้อกันทั้งหมดจะถูกบีบไปกองใกล้ 0 และค่าใน test ที่เกินช่วงของ train จะหลุดนอกช่วง $[0, 1]$ ขณะที่ Z-score ใช้ mean และ std ซึ่งได้รับผลจากค่าสุดโต่งน้อยกว่า และให้ทุกตัวแปรมี mean 0 เหมาะกับ PCA และ penalty ของ Lasso/Ridge (ข้อนี้เป็นเหตุผลเชิงหลักการ ไม่ได้ทดลองเทียบกันด้วย CV)

6. Feature selection

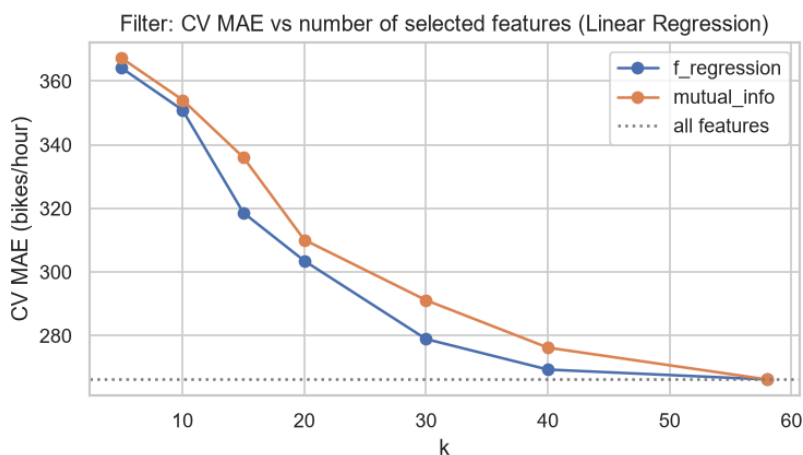
► วิธีดำเนินการ

ทำ 3 แนวทาง โดย selector อยู่ในช่อง `select` ของ Pipeline และเลือกจำนวน feature ด้วย `GridSearchCV + GroupKFold` ประเมินด้วย Linear Regression (hour one-hot, 58 คอลัมน์) เพราะ feature selection มีผลกับโมเดลเชิงเส้นมากที่สุด หมายเหตุ: ใช้ χ^2 ไม่ได้ เพราะ χ^2 ต้องการ target เป็นหมวดหมู่ (และต้องการค่า feature ไม่ติดลบ ขณะที่อุณหภูมิติดลบได้) จึงใช้ `f_regression` และ `mutual_info_regression` แทน

► ผลลัพธ์

ตารางที่ 6.1 สรุปผล feature selection (CV MAE, คำน/ชม.)

วิธี	แนวทาง	พารามิเตอร์ที่ดีที่สุด	จำนวน feature	CV MAE
ไม่เลือก (ทุก feature)	—	—	58	266.2
SelectKBest(f_regression)	Filter	k = 58	58	266.2
SelectKBest(mutual_info)	Filter	k = 58	58	266.2
RFE (จัดอันดับด้วย Ridge)	Wrapper	n = 30	30	265.8
SelectFromModel(Lasso)	Embedded	alpha = 0.1	52	266.2



รูปที่ 6.1 CV MAE ตามจำนวน feature ที่ filter เลือก (k)

ตารางที่ 6.2 CV MAE (คั่น/ชม.) ของ RFE ตามจำนวน feature ที่เก็บ (n) · RFE ตัดตัวที่ |coefficient| เล็กสุดทีละ 2 ตัว

n	5	10	15	20	30	40	58
CV MAE	485.8	313.6	296.9	285.1	265.8	266.0	266.2

ตารางที่ 6.3 Lasso ตามค่า alpha · ยิ่ง alpha สูง ยิ่งดัน coefficient เป็น 0 มากขึ้น

alpha	0.1	0.3	1	3	10	30
feature ที่เหลือ	52	52	48	39	26	7
CV MAE	266.2	266.2	266.4	267.0	274.7	345.9

► การวิเคราะห์และการตัดสินใจ

- **Filter:** MAE ลดลงเรื่อยๆ เมื่อ k เพิ่ม และดีที่สุดเมื่อใช้ทุกตัว เพราะ filter ให้คะแนนทีละคอลัมน์ แต่ one-hot ของชั่วโมงมีประโยชน์ *เมื่ออยู่ด้วยกัน* · ทั้งสองวิธีจัด temp_c เป็นอันดับ 1 และ mutual information จับผลของฝน (is_raining, rainfall_mm) ได้ดีกว่า F-test
- **Wrapper (RFE):** ใช้ 30 จาก 58 feature ได้ MAE เท่าเดิม (ต่าง 0.4 คั่น น้อยกว่า std 4.8) — ตัดข้อมูลซ้ำซ้อนทิ้ง เช่น is_weekend / is_holiday กับ one-hot ของวัน, เดือนกับฤดู และ rainfall_mm (เก็บ is_raining)
- **Embedded (Lasso):** ที่ alpha = 0.1 ดัน coefficient เป็น 0 ให้ one-hot ที่ซ้ำกับกลุ่มอื่น 6 ตัว (day_of_week_3, day_of_week_5, month_4, month_7, month_9, seasons_Summer) โดย MAE ไม่เปลี่ยน แต่ถ้า alpha สูงเกินไปจะตัดตัวที่มีประโยชน์ทิ้งและ MAE แยก (ตารางที่ 6.3) · coefficient ใหญ่สุดเป็นชั่วโมงพิก (hour_18 +836.8, hour_19 +550.5, hour_8 +496.6 และ hour_4 -401.9 คั่น) · ต่างจากที่คาดไว้ Lasso ไม่ได้ตัด dew_point_c ทั้งที่ r กับ temp_c สูง

ตัดสินใจ: ไม่มีวิธีไหนทำให้ MAE ดีขึ้นเกินระดับ noise จึงใช้ทุก feature ในโมเดลสุดท้าย และใช้ผลชั้นนี้อธิบายความสำคัญและความซ้ำซ้อนของ feature

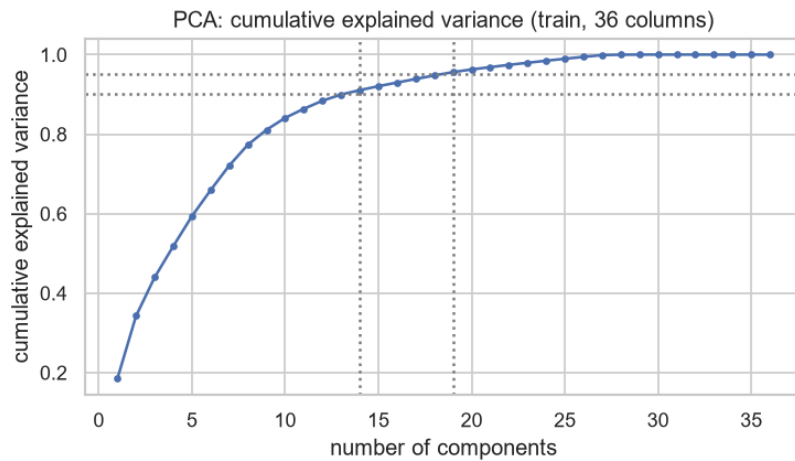
7. PCA

► วิธีดำเนินการ

ทำ PCA บนข้อมูลหลัง standardize (36 คอลัมน์, hour แบบ sin/cos) เพื่อดูว่าบีบมิติได้แค่ไหน ดีความ component แรกๆ จาก loading แล้วใส่ PCA ในช่อง select ของ Pipeline เทียบ CV MAE กับการไม่ใส่ ทั้งใน Linear Regression และ kNN

► ผลลัพธ์

ต้องใช้ **14 components** จึงอธิบาย variance ได้ $\geq 90\%$ และ 19 components สำหรับ $\geq 95\%$ → ข้อมูลซ้ำซ้อนพอสมควรแต่บีบมิติได้ไม่มาก



รูปที่ 7.1 cumulative explained variance

ตารางที่ 7.1 การตีความ principal component แรกๆ จาก loading

PC	variance	loading หลัก	ตีความ
PC1	18.5%	humidity +0.53, dew point +0.39, visibility -0.32, is_raining +0.31	ความชื้น / อากาศมัว
PC2	15.8%	temp +0.56, dew point +0.45, solar +0.44, hour_cos -0.30	ความอุ่น / กลางวัน
PC3	9.7%	rainfall +0.51, is_raining +0.49, hour_cos -0.36	ฝน

ตารางที่ 7.2 CV MAE (ต้น/ชม.) เมื่อใส่ PCA ใน Pipeline เทียบกับไม่ใส่

โมเดล	ไม่มี PCA	5 PC	10 PC	15 PC	20 PC	25 PC
Linear Regression	314.6	364.9	340.2	325.6	329.2	315.6
kNN (k = 10)	194.4	200.8	190.5	189.5	201.3	195.0

► การวิเคราะห์และการตัดสินใจ

PCA **ไม่ช่วย** Linear Regression (ตัดข้อมูลบางส่วนทิ้ง ส่วนโมเดลเชิงเส้นบนแกนที่หมุนแล้วก็ยังเป็นเชิงเส้นเหมือนเดิม) และ **ช่วย kNN เล็กน้อย** (~5 ต้น) เพราะการตัดมิติที่เป็น noise ทำให้ "เพื่อนบ้าน" มีความหมายขึ้น → ใช้ PCA(15) เป็นตัวเลือกหนึ่งใน tuning ของ kNN

8. Baseline และการเทียบโมเดล

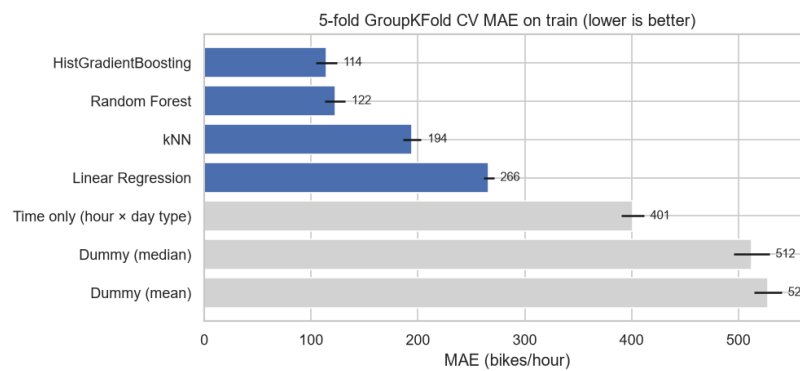
► วิธีดำเนินการ

สร้าง baseline 3 แบบเป็นเพดานล่าง (หาค่าเฉลี่ย, หายมัธยฐาน และค่าเฉลี่ยของแต่ละชั่วโมง × ประเภทวัน) แล้วเทียบ 4 ตระกูลโมเดลที่มีสมมติฐานต่างกัน — เส้นตรง (Linear Regression), ระยะทาง (kNN), ต้นไม้แบบ bagging (Random Forest) และต้นไม้แบบ boosting (HistGradientBoosting) — ด้วยค่า default ภายใต้ 5-fold GroupKFold บน train และบันทึก MAE บน train คู่กันเพื่อดู overfit/underfit ($\text{gap} = \text{CV MAE} - \text{train MAE}$)

► ผลลัพธ์

ตารางที่ 8.1 baseline และ 4 โมเดลด้วยค่า default (5-fold GroupKFold บน train, หน่วยคั่น/ชม. ยกเว้น MSE เป็นคั่น²)

โมเดล	CV MAE	± std	MSE	RMSE	R ²	MAE บน train	gap
Dummy (mean)	527.5	12.9	423,381	650.4	-0.003	527.1	—
Dummy (median)	512.2	16.6	454,336	673.6	-0.075	511.8	—
เวลาอย่างเดียว (hour × ประเภทวัน)	401.0	10.5	270,794	520.2	0.358	396.0	—
Linear Regression (hour one-hot)	266.2	4.8	120,964	347.7	0.713	259.8	6.4
kNN (k = 10, sin/cos)	194.4	8.6	84,316	289.9	0.800	151.9	42.5
Random Forest (200 ต้น)	122.4	9.4	40,192	199.4	0.904	36.8	85.6
HistGradientBoosting	114.1	10.0	34,462	184.6	0.918	69.0	45.1



รูปที่ 8.1 CV MAE ± std (สีเทา = baseline)

► การวิเคราะห์และการตัดสินใจ

- baseline "เวลาอย่างเดียว" คือค่าเฉลี่ยของแต่ละ (ชั่วโมง × ประเภทวัน) ที่เรียนจาก training fold ใช้วัดว่า ข้อมูลอากาศ ช่วยจริงไหม — ทุกโมเดลชนะ baseline นี้ชัดเจน
- **Linear Regression** gap แค่ 6.4 คั่น = underfit (จับความสัมพันธ์โค้งและ interaction ไม่ได้) · **Random Forest** MAE บน train แค่ 36.8 และ gap 85.6 = overfit มากที่สุด (bias ต่ำ variance สูง) · **HistGradientBoosting** แม่นที่สุดและ gap ปานกลาง
- ลำดับ เส้นตรง → ระยะทาง → ต้นไม้ ตรงกับที่ EDA คาดไว้ (ความสัมพันธ์โค้ง + interaction ชั่วโมง × วัน)

9. Hyperparameter tuning

► วิธีดำเนินการ

tune ด้วย `GridSearchCV` (Linear, kNN) และ `RandomizedSearchCV` (RF สุ่ม 15 จาก 64 ชุด, HGB สุ่ม 30 จาก 480 ชุด) ทั้งหมดใช้ `GroupKFold` + `groups` และเลือกด้วย MAE · ลองฝึกบน `log1p(y)` ผ่าน `TransformedTargetRegressor` ด้วย (แปลงค่าทำนายกลับก่อนคิดคะแนน MAE จึงยังเป็นหน่วยคั่น)

► ผลลัพธ์

ตารางที่ 9.1 CV MAE ก่อนและหลัง tune (คั่น/ชม.)

โมเดล	ก่อน	หลัง	พารามิเตอร์ที่ได้
Linear Regression	266.2	266.2	ไม่มีไฮเปอร์พารามิเตอร์
Linear Regression + $\log_{10}(y)$	—	211.0 $\pm$ 6.9	ดีขึ้น 55 คั่น (-21%)
kNN	194.4	183.7 $\pm$ 7.8	k = 5, weights = distance, PCA(15)
kNN + $\log_{10}(y)$	—	188.7 $\pm$ 9.3	k = 15, distance
Random Forest	122.4	121.8 $\pm$ 9.1	200 ต้น, min_samples_leaf = 1, max_features = 0.75
HistGradientBoosting	114.1	109.0 $\pm$ 8.1	learning_rate = 0.02, max_iter = 400, max_leaf_nodes = 127, min_samples_leaf = 5, l2 = 1.0
HistGradientBoosting + $\log_{10}(y)$	—	110.7 $\pm$ 8.0	พารามิเตอร์เดียวกับ HGB

► การวิเคราะห์และการตัดสินใจ

- **$\log_{10}$** ช่วยโมเดลเชิงเส้นมาก เพราะผลของอากาศเป็นแบบ "คูณ" (ฝนตก $\rightarrow$ ยอดลด $\sim 80\%$ ไม่ว่าจะยอดปกติจะเป็นหลัก ร้อยหรือหลักพัน) บนสเกล log ผลแบบคูณกลายเป็นแบบบวกซึ่งโมเดลเชิงเส้นจับได้ · แต่ไม่ช่วย kNN และ HGB ซึ่งไม่ได้สมมติรูปร่างความสัมพันธ์อยู่แล้ว
- พารามิเตอร์ที่ดีที่สุดของ HGB อยู่ที่ขอบของ grid (learning_rate ต่ำสุด, max_leaf_nodes สูงสุด) ขยาย grid ได้อีก แต่ส่วนที่เหลือน่าจะน้อยกว่า std (~ 8 คั่น)

10. การเลือกโมเดลและทรัพยากรที่ใช้

► วิธีดำเนินการ

นำตัวที่ดีที่สุดของแต่ละตระกูลหลัง tune มาเทียบกันทั้งความแม่นยำ (CV MAE) และทรัพยากร (เวลา fit บน train ทั้งหมด, เวลาทำนาย 1 แถว) โดยเลือกด้วย CV ไม่ใช่ test เพื่อให้ test เป็นการประเมินที่ไม่ลำเอียงจริง

► ผลลัพธ์

ตารางที่ 10.1 เปรียบเทียบความแม่นยำและทรัพยากร (ตัวที่ดีที่สุดของแต่ละตระกูลหลัง tune)

โมเดล	CV MAE	fit ทั้ง train (s)	latency / 1 แถว (ms)	ต้นทุนการทำนาย
HistGradientBoosting	109.0 $\pm$ 8.1	5.26	14.1	O(จำนวนต้น $\times$ ความลึก)
Random Forest	121.8 $\pm$ 9.1	3.77	11.7	O(จำนวนต้น $\times$ ความลึก)
kNN (+PCA 15)	183.7 $\pm$ 7.8	0.03	3.9	O(n-d) ต่อครั้ง ต้องเก็บ train ทั้งหมด
Linear Regression + $\log_{10}(y)$	211.0 $\pm$ 6.9	0.04	4.7	O(d)

เวลา fit และ latency เปลี่ยนเล็กน้อยทุกครั้งที่รันและขึ้นกับเครื่อง ใช้ดูลำดับขนาดเท่านั้น (latency = มาตรฐานของการทำนาย 1 แถว 30 ครั้ง รวม preprocessing ใน Pipeline)

► การวิเคราะห์และการตัดสินใจ

เลือก **HistGradientBoosting**: MAE ต่ำที่สุด ห่างจาก Random Forest 12.8 คั่น (~1.5 เท่าของ std จึงไม่ถือว่าเสมอกัน) และทำนาย 1 แถวใช้ไม่กี่มิลลิวินาที · ทรัพยากรไม่ใช่ตัวตัดสิน เพราะทุกโมเดล fit ได้ในไม่กี่วินาทีและทำนายในระดับมิลลิวินาทีใช้งานจริงได้ทั้งหมด · Random Forest ตกไปเพราะแม่นยำน้อยกว่า

11. ผลบนชุดทดสอบและ error analysis

► วิธีดำเนินการ

นำ HistGradientBoosting ที่ fit บน train ทั้งหมดมาทำนาย test 1,697 แถว เป็นครั้งแรกและครั้งเดียว (ค่าทำนายติดลบ 3 แถว clip เป็น 0) แล้ววิเคราะห์ error แยกตามฤดู ประเภทวัน ฝน และชั่วโมง พร้อมดู 10 ชั่วโมงที่หายพลาดมากที่สุด

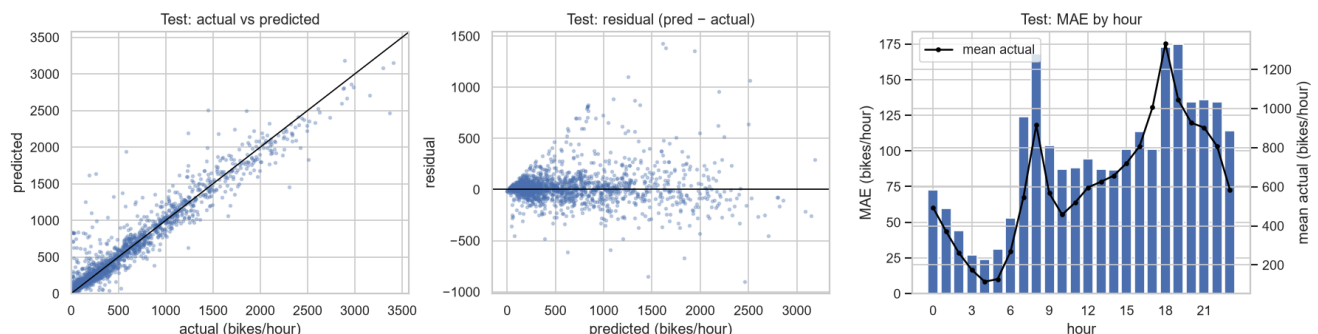
► ผลลัพธ์

ตารางที่ 11.1 CV เทียบ test

เมตริก	CV บน train	test
MAE (คั่น/ชม.)	109.0	97.4
MSE (คั่น ²)	—	28,947.7
RMSE (คั่น/ชม.)	—	170.1
R ²	—	0.919

ตารางที่ 11.2 error แยกกลุ่ม (bias = ค่าเฉลี่ยของ ทำนาย - จริง, บวก = หายสูงเกิน)

กลุ่ม	n	ยอดเฉลี่ย	MAE	bias	MAE / ยอดเฉลี่ย
Winter	552	211.7	49.7	+5.5	23.5%
Spring	336	596.0	97.0	+30.3	16.3%
Autumn	425	805.5	126.4	+49.7	15.7%
Summer	384	1,018.1	134.3	+20.6	13.2%
วันทำงาน	1,128	681.3	102.6	+18.6	15.1%
วันหยุด	569	495.4	87.2	+37.4	17.6%
ฝนตก	103	180.2	159.8	+71.2	88.7%
ไม่มีฝน	1,594	647.3	93.4	+21.9	14.4%



รูปที่ 11.1 ค่าจริง vs ค่าทำนาย (ซ้าย), residual (กลาง), MAE รายชั่วโมงเทียบยอดเฉลี่ย (ขวา)

ตารางที่ 11.3 10 ชั่วโมงที่หายพลาดมากที่สุดใน test (error = ทำนาย - จริง, บวก = หายเกิน)

วันที่	ชม.	ฤดู	°C	ความชื้น %	ฝน มม.	ประเภทวัน	จริง	ทำนาย	error
2018-08-23 (พฤ.)	21	Summer	28	72	0	วันทำงาน	189	1,613	+1,424
2018-08-23 (พฤ.)	20	Summer	29	62	0	วันทำงาน	257	1,642	+1,385
2018-08-23 (พฤ.)	19	Summer	30	59	0	วันทำงาน	582	1,939	+1,357
2018-11-08 (พฤ.)	8	Autumn	11	89	0	วันทำงาน	148	1,249	+1,101
2018-08-23 (พฤ.)	18	Summer	31	55	0	วันทำงาน	1,447	2,510	+1,063
2018-09-24 (จ.)	18	Autumn	20	35	0	วันหยุด	1,237	2,193	+956
2018-06-22 (ศ.)	18	Summer	29	27	0	วันทำงาน	3,365	2,464	-901
2018-08-23 (พฤ.)	22	Summer	27	84	2	วันทำงาน	215	1,102	+887
2018-07-09 (จ.)	7	Summer	21	66	0	วันทำงาน	451	1,323	+872
2018-06-12 (อ.)	22	Summer	20	70	0	วันทำงาน	2,309	1,455	-854

► การวิเคราะห์และการตัดสินใจ

- test MAE ใกล้กับ CV แปลว่าโมเดลไม่ได้ overfit และ CV ประเมินได้สมจริง ที่ test ต่ำกว่า CV ~12 คั่น สอดคล้องกับที่บันทึกไว้ในหัวข้อ 3: test มีชั่วโมงฤดูหนาวมากกว่า ซึ่งยอดต่ำ ค่าคลาดเป็นคั่นจึงเล็กตาม ไม่ได้แปลว่าโมเดลเก่งขึ้น
- ค่าทำนายเกาะเส้นแทบดี แต่ **ความคลาดโตตามระดับยอด** (residual เป็นรูปกรวย) · MAE สูงสุดที่ชั่วโมงพีค 18:00–19:00 และ 8:00 ต่ำสุดดี 3–5
- **ชั่วโมงฝนตกคือจุดอ่อนที่สุด**: คลาด 88.7% ของยอดและหายสูงเกิน (ชั่วโมงฝนตกมีแค่ ~6% ของ train ให้เรียน)
- bias เป็นบวกทุกกลุ่ม → โดยรวมโมเดลหายสูงกว่าจริงเล็กน้อย
- ใน 10 ชั่วโมงที่พลาดมากที่สุด (ตารางที่ 11.3) หายเกิน 8 แถว และทุกแถวเป็นชั่วโมงเดินทาง (7:00–8:00 และ 18:00–22:00) ซึ่งยอดสูง ค่าคลาดเป็นคั่นจึงใหญ่ตาม
- **10 ชั่วโมงที่ผิดมากที่สุด 5 ชั่วโมงมาจากวันเดียว** คือ 23 ส.ค. 2018 เวลา 18:00–22:00 ซึ่งอากาศในข้อมูลดูปกติแต่ยอดจริงต่ำมาก (เช่น 21:00 จริง 189 หาย 1,613) วันนั้นพายุไต้ฝุ่น Soulik ขึ้นฝั่งเกาหลีใต้ อีกกรณีคือ 24 ก.ย. 2018 ช่วงเทศกาลชูปะชาที่คนเดินทางออกจากโซล → error ก้อนใหญ่มาจาก **เหตุการณ์ที่ไม่มีใน feature**

12. การทำนายข้อมูลใหม่

► วิธีดำเนินการ

สร้างสถานการณ์สมมติในรูปแบบเดียวกับข้อมูลดิบ ผ่านฟังก์ชันสร้าง feature ชุดเดียวกับ train (dew point คำนวนจากอุณหภูมิและความชื้นให้สอดคล้องกันทางกายภาพ) แล้วทำนายด้วยโมเดลที่เลือก

Pipeline ที่ใช้ทำนายจริง: ข้อมูลดิบ 1 แถว (วันที่ ชั่วโมง อากาศ ฤดู วันหยุด) ของชั่วโมงที่ระบบเปิด → `add_features()` ฟังก์ชันเดียวกับตอนฝึก ได้ 17 คอลัมน์ (FEATURES) → `final_model` ซึ่งรวม `SimpleImputer(median)` → `StandardScaler` สำหรับตัวแปรตัวเลข และ `OneHotEncoder` สำหรับฤดู เดือน วันในสัปดาห์ วั้หน้า `HistGradientBoostingRegressor` → ตัดค่าทำนายที่ติดลบเป็น 0 · ค่าที่ใช้แปลงทุกตัว (median, mean/std, หมวดหมู่) เรียนจาก train เท่านั้นและติดมากับ `final_model` ตอนทำนายจึงไม่ต้อง fit อะไรใหม่

► ผลลัพธ์

ตารางที่ 12.1 ผลทำนายสถานการณ์สมมติ

สถานการณ์	ทำนาย (คัน/ชม.)	ความหมาย
พุธ 18:00 ฤดูร้อน 24 °C ไม่มีฝน	2,981	พักผ่อนวันทำงาน + อากาศดี = ต้องเตรียมจักรยานมากที่สุด
พุธ 18:00 เดียวกัน แต่ฝนตก 3 มม. (20 °C, ชื้น 95%)	204	ฝนทำให้ยอดขายไป ~93%
เสาร์ 18:00 อากาศเหมือนแถวแรก	2,167	วันหยุดไม่มีพักเดินทางกลับบ้าน ต่ำกว่าวันพุธ ~800 คัน
พุธ 08:00 ฤดูหนาว -3 °C	690	ยังมีพักเช้า แต่ต่ำกว่าฤดูร้อนมาก

► การวิเคราะห์และการตัดสินใจ

ทิศทางของทุกปัจจัยสมเหตุสมผลและตรงกับ EDA แต่ค่าทำนายของชั่วโมงฝนตกมีความไม่แน่นอนสูงที่สุด (หัวข้อ 11)

13. ความท้าทายที่พบ

ปัญหา	วิธีแก้
ชั่วโมงในวันเดียวกันคล้ายกันมาก (leakage แบบซ่อน)	แบ่ง train/test และทำ CV ตามวันด้วย GroupShuffleSplit / GroupKFold และแสดงให้เห็นว่า KFold ธรรมดาทำให้ทุกวันใน validation รั่วไปอยู่ใน train
ชั่วโมงที่ระบบปิด ยอด = 0 เสมอ	ตัดออกและประกาศขอบเขตของโมเดล แทนที่จะปล่อยให้คะแนนขึ้นแบบไม่มีความหมาย
ไฟล์ไม่ใช่ UTF-8 และวันที่กำกวม	ตรวจ byte ดิบ เลือก cp1252 และระบุรูปแบบวันที่เอง
ความชื้น 0% (ค่าผิดของเซนเซอร์)	ยืนยันด้วยสูตร Magnus แล้วแปลงเป็น NaN + เติมค่าใน Pipeline
chi ² ใช้กับ regression ไม่ได้ (และต้องการค่าไม่ติดลบ)	ใช้ f_regression และ mutual_info_regression
one-hot ของเดือน/ฤดู/วันซ้อนกัน (multicollinearity)	ใช้ Ridge จัดอันดับใน RFE และ Lasso ตัด one-hot ที่ซ้ำ
ชั่วโมงมี 2 พิก แต่ sin/cos ให้โมเดล เชิงเส้นได้แค่ 1 พิก	เลือก encoding ของชั่วโมงตามชนิดโมเดล

14. อภิปรายผล ข้อจำกัด และแนวทางพัฒนาต่อ

► อภิปรายผล

- ผลสอดคล้องกับที่ EDA คาดไว้: ชั่วโมงและอุณหภูมิสัมพันธ์กับยอดขายแบบไม่เป็นเส้นตรงและมี interaction กับประเภทวัน โมเดลต้นไม้จึงแม่นยำกว่าโมเดลเชิงเส้นมาก (HistGradientBoosting 109.0 เทียบ Linear Regression 266.2 คัน/ชม.)
- feature selection และ PCA ไม่ได้ทำให้แม่นยำขึ้นเกินระดับ noise แต่ยืนยันว่ามี feature ซ้ำซ้อน (RFE ใช้ 30 จาก 58 คอลัมน์ได้ MAE เท่าเดิม) ซึ่งมีประโยชน์ถ้าต้องการโมเดลที่เล็กลง
- test MAE (97.4) ต่ำกว่า CV (109.0) เพราะ test มีชั่วโมงฤดูหนาวมากกว่า ไม่ได้แปลว่าโมเดลเก่งขึ้น จึงต้องอ่านคู่กับ MAE แยกฤดู (ตารางที่ 11.2)
- error ก้อนใหญ่ที่สุดมาจากเหตุการณ์ที่ข้อมูลไม่มี เช่น ใต้ฝุ่นและเทศกาล (ตารางที่ 11.3) มากกว่าจากการเลือกโมเดล

► ข้อจำกัดและแนวทางพัฒนาต่อ

- ชั่วโมงที่ฝนตก คลาดมาก → เพิ่มข้อมูลหลายปี หรือถ่วงน้ำหนักชั่วโมงฝนตกตอนฝឹก
- เหตุการณ์พิเศษ (พายุ, เทศกาล, ประกาศเตือนภัย) ไม่อยู่ใน feature → เพิ่ม feature ปฏิทินเทศกาลหรือค่าเตือนสภาพอากาศ
- ข้อมูลมีแค่ 1 ปีและเป็นยอดรวมทั้งเมือง ใช้วางแผนรายสถานีไม่ได้ และใช้ได้เฉพาะชั่วโมงที่ระบบเปิด
- split แบบสุ่มวันวัดความสามารถ "อากาศ + เวลา → ยอดเช่า" ไม่ได้วัดการพยากรณ์ล่วงหน้าหลายเดือน
- input ตอนใช้งานจริงคือ พยากรณ์อากาศ ซึ่งคลาดเคลื่อนมากกว่าค่าที่วัดได้จริงในชุดข้อมูล → ความแม่นยำจริงน่าจะต่ำกว่าที่รายงาน
- พารามิเตอร์ของ HGB อยู่ที่ชอบ grid ขยายการค้นหาได้อีก

15. สรุป

HistGradientBoostingRegressor ทำนายยอดเช่าจักรยานรายชั่วโมงได้คลาดเฉลี่ยราว **100 คัน/ชม.** (CV 109.0, test 97.4) อธิบายความแปรปรวนได้ ~92% และลดความคลาดเคลื่อนจากการใช้แค่ปฏิทินได้ ~73% แสดงว่าข้อมูลอากาศมีประโยชน์จริง ปัจจัยที่สำคัญที่สุดคือชั่วโมง (พีคเดินทาง 8:00 และ 18:00 ในวันทำงาน) อุณหภูมิ (ความสัมพันธ์โค้ง) และการมีฝน ผลนี้ได้มาภายใต้การป้องกัน leakage อย่างเข้มงวด (split และ CV ตามวัน, ทุกชั้นที่เรียนจากข้อมูลอยู่ใน Pipeline, เปิด test ครั้งเดียว) จึงเชื่อถือว่าสะท้อนการใช้งานกับวันที่โมเดลไม่เคยเห็น

ภาคผนวก ก. เช็คลิสต์ป้องกัน leakage

ข้อ	สถานะ
split train/test ตามวัน และทุก CV ใช้ GroupKFold + ส่ง groups	ผ่าน
ไม่มี feature ที่คำนวณจาก target (ไม่มี lag หรือค่าเฉลี่ยย้อนเช่า)	ผ่าน
rented_bike_count ไม่อยู่ใน X	ผ่าน
imputer, scaler, encoder, selector, PCA และการแปลง log1p อยู่ใน Pipeline	ผ่าน
การตัดแฉะระบบปิดและการแปลงความชื้น 0% เป็นกฎตายตัว ไม่ได้เรียนค่าจากข้อมูล	ผ่าน
ไม่ได้เปิด test ระหว่าง EDA จนถึงเลือกโมเดล และประเมิน test ครั้งเดียว	ผ่าน

ภาคผนวก ข. การรันซ้ำ

โค้ดทั้งหมดอยู่ใน notebook 3 ไฟล์ รันตามลำดับ 00_setup → 01_eda → 02_modeling บน Python 3.13 ด้วยไลบรารีตาม requirements.txt (scikit-learn 1.9.1, pandas 2.3.3) · ทุกจุดที่มีการสุ่มใช้ random_state = 42 · เมื่อรันซ้ำบนเครื่องใหม่ได้ MAE, RMSE, R² และพารามิเตอร์ที่ดีที่สุดตรงกันทุกค่า ต่างกันแค่เวลา fit/latency · โมเดลสุดท้ายไม่ได้บันทึกเป็นไฟล์ 02_modeling ชั้น 12 สร้าง final_model ในหน่วยความจำแล้วใช้ต่อในชั้น 13 (รันซ้ำได้โมเดลเดิมเพราะตั้ง random_state) · รายงานนี้สร้างจาก reports/report.html ด้วย python reports/build_pdf.py

1. Seoul Bike Sharing Demand [Dataset]. (2020). UCI Machine Learning Repository. <https://doi.org/10.24432/C5F62R> (CC BY 4.0)
2. Sathishkumar V E, Park J., Cho Y. (2020). Using data mining techniques for bike sharing demand prediction in metropolitan city. *Computer Communications*, 153, 353–366.
3. Pedregosa F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
4. Korea Times (2018-08-24). Typhoon Soulik weakens after storming southern areas. <https://www.koreatimes.co.kr/southkorea/society/20180824/typhoon-soulik-weakens-after-storming-southern-areas>
5. Wikipedia. Chuseok. <https://en.wikipedia.org/wiki/Chuseok>